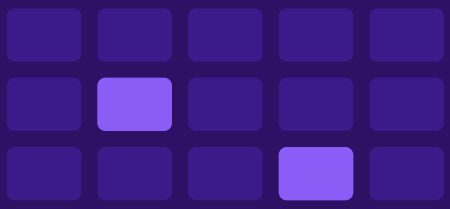
INGENIERÍA EN INFORMÁTICA Y SISTEMAS



Sistema Inteligente de Gestión de Parqueos

Distribución de tareas y especificación de integración

Frontend React + Vite · API Gateway · Parking Service
Reservation Service · Payment Service · Supabase PostgreSQL
Monorepo en GitHub · Podman Compose

Documento de trabajo del equipo · Prototipo universitario

Índice

1. Propósito, contexto y alcance **3**

1.1. Arquitectura . . . 3

1.2. Fuera del alcance . . . 3

1.3. Estado actual del proyecto . . . 3

2. Distribución de responsabilidades **4**

2.1. Tabla general . . . 4

2.2. Frontend . . . 4

2.3. Parking Service . . . 5

2.4. Reservation Service . . . 6

2.5. Payment Service . . . 7

2.6. API Gateway e integración . . . 8

3. Contrato de integración (obligatorio) **9**

3.1. Convenciones generales . . . 9

3.2. Nombres oficiales de los campos JSON . . . 9

3.3. Estados oficiales del sistema . . . 11

3.4. Formato uniforme de las respuestas HTTP . . . 11

3.5. Códigos HTTP obligatorios . . . 12

3.6. Contratos oficiales de los endpoints . . . 13

3.7. Rutas públicas frente a rutas internas . . . 15

4. Flujo oficial de integración **16**

4.1. Flujo de salida y pago . . . 16

4.2. Fallos parciales . . . 16

4.3. Flujo de prueba completo . . . 16

5. Distribución de la base de datos **16**

6. Infraestructura y tareas compartidas **18**

7. Orden recomendado de desarrollo **18**

8. Entregables de cada responsable **19**

9. Adaptaciones pendientes respecto al código actual **19**

Cómo usar este documento. Cada integrante debe leer la sección 1, la sección 3 (contrato de integración) y la sección de su componente en la sección 2. El contrato de la sección 3 es definitivo: si una implementación existente difiere, se adapta la implementación, no el contrato.

1. Propósito, contexto y alcance

Este documento reparte las tareas de desarrollo del **Sistema Inteligente de Gestión de Parqueos**, un proyecto universitario de Ingeniería en Informática y Sistemas. Su objetivo es que cada integrante sepa qué componente le corresponde, en qué carpeta trabaja, qué endpoints debe implementar y cómo se comunica su componente con los demás.

1.1. Arquitectura

Capa	Tecnología	Notas
Frontend	React + Vite	Consume todo el backend únicamente a través del API Gateway.
API Gateway	Node.js + Express	Punto de entrada único; enruta hacia los microservicios.
Microservicios	Node.js + Express	Parking, Reservation y Payment Service, cada uno con su esquema de datos.
Base de datos	Supabase PostgreSQL	Un solo proyecto con tres esquemas lógicos.
Comunicación	HTTP + JSON	Formato uniforme definido en la sección 3.
Contenedores	Podman Compose	Orquesta todos los servicios.
Repositorio	Monorepo en GitHub	Una carpeta por componente.

La aplicación debe permitir consultar plazas, reservar una plaza, registrar la entrada y la salida de vehículos, calcular el importe del estacionamiento y registrar pagos simulados.

1.2. Fuera del alcance

- Sensores físicos y cámaras.
- Pagos reales: no se procesa dinero real, tarjetas ni transferencias.
- Cualquier funcionalidad fuera del alcance del prototipo.
- Convertir el proyecto en un monolito: la arquitectura de microservicios se mantiene.

1.3. Estado actual del proyecto

> Solo se ha verificado lo siguiente: el **listado de plazas** y la **conexión con Supabase** ya se probaron; la tabla `parking.parking_spaces` ya existe; el Gateway cuenta con el endpoint `/health`. Todo lo demás es **tarea pendiente** hasta que el responsable lo implemente y lo verifique. Antes de crear rutas o tablas, revisar el código existente para no duplicar componentes.

Elemento	Estado
Listado de plazas (Parking Service)	Probado previamente. **Requiere adaptación** al formato de respuesta oficial (sección 3.4).
Conexión con Supabase	Probada previamente.
Tabla `parking.parking_spaces`	Existe. Revisar que sus columnas coincidan con el contrato.
Endpoint `/health` del Gateway	Existente según el equipo; mantenerlo.
Resto de endpoints, tablas y pantallas	**Pendiente.** No se afirma que existan hasta verificarlos.

2. Distribución de responsabilidades

2.1. Tabla general

Área	Carpeta	Tecnología	Esquema de datos
Frontend	`frontend/`	React + Vite	Ninguno (no accede a Supabase)
Parking Service	`parking-service/`	Node.js + Express	`parking`
Reservation Service	`reservation-service/`	Node.js + Express	`reservations`
Payment Service	`payment-service/`	Node.js + Express	`payments`
API Gateway e integración	`api-gateway/`	Node.js + Express	Ninguno (no accede a PostgreSQL)

Equipos con menos integrantes que áreas. Se pueden combinar funciones, pero cada componente debe tener un responsable definido por nombre antes de empezar a programar.

Rutas públicas e internas. Las rutas con prefijo `/api/...` son públicas y se consumen a través del Gateway. Las rutas internas son las que expone cada microservicio dentro de la red de contenedores; solo el Gateway (y los otros servicios, cuando el flujo lo exige) deben llamarlas.

2.2. Frontend

Integrante responsable del frontend

Carpeta: `frontend/` | Tecnología: React + Vite

Responsabilidades

- ☐ Crear un panel principal con las cantidades de plazas disponibles, reservadas y ocupadas.
- ☐ Mostrar una cuadrícula o mapa visual de plazas.
- ☐ Mostrar código, nivel, tipo y estado de cada plaza.
- ☐ Crear formularios para reservar plazas.
- ☐ Crear pantallas para registrar entrada y salida de vehículos.
- ☐ Mostrar la duración de una sesión de estacionamiento.
- ☐ Mostrar el importe calculado y permitir registrar un pago simulado.
- ☐ Mostrar confirmaciones, errores y resultados de las operaciones.
- ☐ Centralizar las llamadas HTTP en una carpeta de servicios.
- ☐ Generar un `Idempotency-Key` (UUID) por cada intento de pago y reutilizarlo en los reintentos.
- ☐ Dividir `amountMinor` entre 100 solo para mostrar el importe (por ejemplo, 1500 se muestra como Q15.00).

Restricciones: no conectar el frontend directamente a Supabase; consumir todas las funcionalidades del backend mediante el API Gateway; usar exactamente los nombres de campos camelCase del contrato.

ESTRUCTURA INICIAL SUGERIDA

```
frontend/src/
├── components/
├── pages/
├── services/
│   ├── api.js
│   ├── parking.service.js
│   ├── reservation.service.js
│   └── payment.service.js
└── App.jsx
```

Estado: pendiente.

2.3. Parking Service

Integrante responsable de Parking Service

Carpeta: `parking-service/` | Tecnología: Node.js + Express

Responsabilidades

- ☐ Consultar todas las plazas.
- ☐ Consultar una plaza por ID (validar que sea UUID antes de consultar la base de datos).
- ☐ Registrar plazas nuevas (`code`, `level`, `spaceType`).
- ☐ Actualizar el estado de una plaza mediante operaciones autorizadas.
- ☐ Validar datos de entrada y gestionar errores de base de datos sin exponer detalles internos.
- ☐ Validar las transiciones de estado en el backend y evitar las incompatibles con reservas activas.
- ☐ Mantener sus datos en el esquema PostgreSQL `parking`.
- ☐ Adaptar el listado existente al formato oficial (`data` + `meta.total`, campos camelCase).

Endpoints

Método	Ruta pública (vía Gateway)	Ruta interna	Función
GET	`/api/parking/spaces`	`/spaces`	Listar plazas
GET	`/api/parking/spaces/:id`	`/spaces/:id`	Consultar una plaza
POST	`/api/parking/spaces`	`/spaces`	Registrar una plaza
PATCH	`/api/parking/spaces/:id/status`	`/spaces/:id/status`	Actualizar estado

Estado: el listado de plazas y la conexión con Supabase ya se han probado. El responsable debe **revisar el código existente antes de duplicar funcionalidades** y adaptarlo al contrato oficial (por ejemplo, el campo `space_type` pasa a `spaceType` en JSON). El resto de endpoints está pendiente.

2.4. Reservation Service

Integrante responsable de Reservation Service

Carpeta: `reservation-service/` | Tecnología: Node.js + Express

Responsabilidades

- ☐ Crear reservas (estado inicial `confirmed`).
- ☐ Consultar y listar reservas.
- ☐ Cancelar reservas cuando corresponda (estado `cancelled`).
- ☐ Registrar la entrada de vehículos: crear la sesión con estado `active` y devolver `sessionId`.
- ☐ Registrar la salida de vehículos: cerrar la sesión (`completed`), informar `exitAt` y calcular `durationSeconds`.
- ☐ Evitar sesiones duplicadas y rechazar una segunda entrada o un doble cierre de sesión.
- ☐ Coordinar con Parking Service la validación de disponibilidad y el cambio de estado de la plaza.
- ☐ Evitar reservas simultáneas de una misma plaza.
- ☐ Contemplar fallos parciales: comprobar estados, reintentos seguros y compensaciones.
- ☐ Mantener sus datos en el esquema `reservations`.

Endpoints

Método	Ruta pública (vía Gateway)	Función
POST	`/api/reservations`	Crear reserva
GET	`/api/reservations`	Listar reservas
GET	`/api/reservations/:id`	Consultar reserva
PATCH	`/api/reservations/:id/cancel`	Cancelar reserva
POST	`/api/reservations/sessions/:reservationId/entry`	Registrar entrada (el parámetro es el `reservationId`; devuelve el `sessionId`)
POST	`/api/reservations/sessions/:sessionId/exit`	Registrar salida
GET	`/api/reservations/sessions/:sessionId`	Consultar sesión

Rutas internas propuestas (a confirmar con el responsable del Gateway): `/reservations`, `/reservations/:id`, `/reservations/:id/cancel`, `/sessions/:id/entry`, `/sessions/:id/exit` y `/sessions/:id`. Las rutas públicas de sesiones llevan el segmento `sessions` bajo `/api/reservations`, por lo que el Gateway debe traducirlas explícitamente.

> Los estados de reservas y sesiones **ya están definidos** en la sección 3.3; el responsable debe documentarlos en su README y no agregar estados nuevos (por ejemplo `pending`, `active` u `occupied` en reservas). **Estado:** pendiente.

2.5. Payment Service

Integrante responsable de Payment Service

Carpeta: `payment-service/` | Tecnología: Node.js + Express

Responsabilidades

- ☐ Definir la tarifa por hora (en centavos, moneda `GTQ`) y registrarla en `payments.tariffs`.
- ☐ Aplicar la regla de redondeo: se cobra cada hora iniciada, mínimo una hora para una sesión de duración positiva.
- ☐ Calcular el importe de una sesión finalizada y devolver el detalle del cálculo.
- ☐ Registrar pagos simulados (único método permitido: `simulated`).
- ☐ Consultar el estado de un pago por `id` y por `sessionId`.
- ☐ Exigir la cabecera `Idempotency-Key` y evitar pagos duplicados por reintentos.
- ☐ Validar que la cotización corresponde a la sesión y que no está vencida.
- ☐ Mantener sus datos en el esquema `payments`.

Endpoints

Método	Ruta pública (vía Gateway)	Función
POST	`/api/payments/quote`	Calcular importe
POST	`/api/payments`	Registrar pago simulado (requiere `Idempotency-Key`)
GET	`/api/payments/:id`	Consultar pago
GET	`/api/payments/session/:sessionId`	Consultar pago de una sesión

No se procesa dinero real. El estado `succeeded` solo indica que el sistema confirmó la simulación. La tarifa de Q15.00 por hora que aparece en los ejemplos es ilustrativa y no establece la tarifa definitiva. **Estado:** pendiente.

2.6. API Gateway e integración

Integrante responsable del API Gateway e integración

Carpeta: `api-gateway/` | Tecnología: Node.js + Express

Responsabilidades

- ☐ Mantener el endpoint `/health`.
- ☐ Enrutar `/api/parking/*` a Parking Service.
- ☐ Enrutar `/api/reservations/*` a Reservation Service.
- ☐ Enrutar `/api/payments/*` a Payment Service.
- ☐ Configurar las URL de los servicios mediante variables de entorno.
- ☐ Conservar el formato de respuesta de los microservicios y reenviar la cabecera `Idempotency-Key`.
- ☐ Gestionar errores de comunicación con el formato de error uniforme y los códigos 502, 503 y 504.
- ☐ Probar las rutas integradas y coordinar la integración del frontend con los microservicios.
- ☐ Apoyar en pruebas, documentación e integración con Podman Compose.

Ruta pública	Destino
`/api/parking/*`	Parking Service
`/api/reservations/*`	Reservation Service
`/api/payments/*`	Payment Service
`/health`	Estado del propio Gateway

Límites: el Gateway no debe implementar la lógica de reservas, calcular tarifas ni conectarse directamente a PostgreSQL. **Estado:** `/health` existente; el resto de rutas está pendiente de verificar.

3. Contrato de integración (obligatorio)

> **IMPORTANTE.** Las siguientes convenciones son definitivas. Todos los integrantes deben respetarlas. No cambiar nombres de campos, estados, rutas, tipos de datos ni formatos de respuesta de manera individual. Si una implementación existente difiere, se adapta la implementación al contrato.

Antes de programar en paralelo, el equipo ya dispone de acuerdos sobre: nombres exactos de campos JSON, formato de identificadores, estados válidos, formato de fechas y zona horaria, códigos HTTP, responsabilidad de cada microservicio sobre sus datos y formato uniforme de respuestas. Todo ello se detalla a continuación.

3.1. Convenciones generales

Elemento	Estándar obligatorio
JSON	`camelCase`
Tablas y columnas PostgreSQL	`snake_case`
Identificadores	UUID v4, representados como strings
Fechas y horas	ISO 8601 en UTC, con sufijo `Z`
Idioma de los identificadores técnicos	Inglés
Protocolo	HTTP sobre JSON
Moneda	`GTQ`
Importes monetarios	Enteros en centavos
Placas de vehículos	String, sin asumir un patrón rígido de longitud
Codificación	UTF-8
Versionado	Las rutas actuales no incluyen versión; no agregar `/v1` de forma individual

Ejemplo de UUID: `550e8400-e29b-41d4-a716-446655440000`

Ejemplo de fecha: `2026-10-19T18:30:00.000Z`

- Los identificadores deben validarse antes de realizar operaciones en la base de datos.
- No utilizar números autoincrementales como identificadores públicos de recursos.
- Los importes se expresan en centavos para evitar errores de precisión: `1500` representa Q15.00. El frontend divide el importe entre 100 únicamente para mostrarlo al usuario.

3.2. Nombres oficiales de los campos JSON

Estos son los únicos nombres permitidos para los campos compartidos entre los componentes.

Campo	Tipo	Significado
`id`	string UUID	Identificador de un recurso cuando se consulta directamente
`spaceId`	string UUID	Identificador de una plaza
`reservationId`	string UUID	Identificador de una reserva
`sessionId`	string UUID	Identificador de una sesión de estacionamiento
`paymentId`	string UUID	Identificador de un pago

Campo	Tipo	Significado
`quoteId`	string UUID	Identificador de una cotización de pago
`code`	string	Código visible de una plaza (por ejemplo `A-01`)
`level`	string	Nivel físico de la plaza
`spaceType`	string	Tipo de plaza
`status`	string	Estado actual del recurso
`vehiclePlate`	string	Matrícula del vehículo
`createdAt`	string fecha	Fecha de creación
`updatedAt`	string fecha	Fecha de última actualización
`entryAt`	string fecha o null	Fecha de entrada del vehículo
`exitAt`	string fecha o null	Fecha de salida del vehículo
`durationSeconds`	integer o null	Duración de la sesión en segundos
`billableHours`	integer	Horas cobrables utilizadas en el cálculo
`hourlyRateMinor`	integer	Tarifa por hora en centavos
`amountMinor`	integer	Importe en centavos
`currency`	string	Código ISO de moneda: `GTQ`
`expiresAt`	string fecha o null	Fecha de expiración de una cotización
`method`	string	Método de pago utilizado
`message`	string	Mensaje descriptivo de error
`error.code`	string	Código técnico del error, dentro del objeto `error`
`details`	array, objeto o null	Información adicional sobre el error
`requestId`	string UUID	Identificador de seguimiento de una petición

Reglas importantes

- Para el tipo de plaza utilizar `spaceType`, nunca `space_type` en JSON.
- Para el importe utilizar `amountMinor`, nunca `amount`, `total`, `price` o `totalPrice`.
- Para las fechas utilizar `entryAt` y `exitAt`, nunca variantes como `entryDate`, `startTime` o `exitDate`.
- Para la matrícula utilizar siempre `vehiclePlate`.
- Los campos de fecha sin valor se devuelven como `null`, no como cadenas vacías.
- No devolver contraseñas, credenciales de Supabase ni variables de entorno.

3.3. Estados oficiales del sistema

Recurso (campo)	Valores permitidos	Notas
Plaza (`status`)	`available`, `reserved`, `occupied`, `maintenance`	Flujo normal: `available` → `reserved` → `occupied` → `available`
Reserva (`status`)	`confirmed`, `cancelled`, `completed`	Se crea como `confirmed`. No agregar `pending`, `active` ni `occupied`.
Sesión (`status`)	`active`, `completed`, `cancelled`	Comienza `active`, con `entryAt` informado y `exitAt` en `null`.
Pago (`status`)	`pending`, `succeeded`, `failed`	`succeeded` significa simulación confirmada, no dinero transferido.
Tipo de plaza (`spaceType`)	`standard`, `accessible`, `electric`	
Método de pago (`method`)	`simulated`	Único valor permitido en el prototipo.

Reglas de las plazas

- Una plaza en mantenimiento no puede reservarse ni ocuparse.
- Una plaza ocupada no puede reservarse.
- Una plaza reservada no puede asignarse a otra reserva activa.
- La plaza vuelve a estar disponible cuando termina la sesión y se confirma el cierre de la operación.
- Las transiciones deben validarse en el backend, no únicamente en la interfaz.

Reglas de reservas y sesiones

- Al cancelar una reserva se establece `cancelled`; al finalizar correctamente la operación de estacionamiento se establece `completed`.
- La entrada y la salida se representan mediante una sesión independiente.
- Al registrar la salida, la sesión pasa a `completed`, se informa `exitAt` y se calcula `durationSeconds`.
- No se permite registrar una segunda entrada ni cerrar dos veces una sesión ya finalizada.
- Una sesión puede pasar a `cancelled` si se anula antes de completarse.

3.4. Formato uniforme de las respuestas HTTP

Todos los microservicios deben usar el mismo formato. El Gateway lo conserva sin modificarlo.

RESPUESTA EXITOSA CON UN RECURSO

```
{
  "data": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "status": "available"
  }
}
```

RESPUESTA EXITOSA DE UNA COLECCIÓN

```
{
  "data": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "code": "A-01",
      "level": "Nivel 1",
      "spaceType": "standard",
      "status": "available"
    }
  ],
  "meta": {
    "total": 1
  }
}
```

Las colecciones usan `data` como arreglo y `meta.total` como cantidad de registros. El endpoint de listado de plazas que ya existe **debe adaptarse** a este formato si actualmente devuelve otra estructura (por ejemplo, `total` y `spaces`, o `space_type`).

RESPUESTA DE ERROR

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Los datos enviados no son válidos.",
    "details": [
      {
        "field": "vehiclePlate",
        "message": "La matrícula es obligatoria."
      }
    ],
    "requestId": "550e8400-e29b-41d4-a716-446655440000"
  }
}
```

`details` puede ser `null` cuando no haya información adicional. `requestId` debe permitir rastrear la petición en los registros del servidor.

3.5. Códigos HTTP obligatorios

Código	Uso
`200 OK`	Consulta o actualización exitosa
`201 Created`	Recurso creado correctamente
`204 No Content`	Operación exitosa sin cuerpo de respuesta
`400 Bad Request`	JSON incorrecto o parámetros inválidos
`404 Not Found`	Recurso inexistente
`409 Conflict`	Conflicto de estado, reserva duplicada o pago ya procesado
`422 Unprocessable Entity`	Datos bien formados pero incompatibles con las reglas del negocio
`500 Internal Server Error`	Error inesperado del servidor
`502 Bad Gateway`	El Gateway no pudo obtener una respuesta válida de un servicio
`503 Service Unavailable`	Servicio temporalmente no disponible
`504 Gateway Timeout`	Un servicio excedió el tiempo de espera del Gateway

No devolver errores internos de PostgreSQL, trazas de ejecución ni credenciales al cliente.

Códigos técnicos de error

`VALIDATION_ERROR`, `INVALID_ID`, `RESOURCE_NOT_FOUND`, `SPACE_NOT_AVAILABLE`, `RESERVATION_CONFLICT`, `INVALID_STATE_TRANSITION`, `SESSION_ALREADY_COMPLETED`, `PAYMENT_ALREADY_PROCESSED`, `DATABASE_ERROR`, `SERVICE_UNAVAILABLE` e `INTERNAL_ERROR`.

3.6. Contratos oficiales de los endpoints

Todas las rutas con prefijo `/api/...` son públicas y se consumen a través del API Gateway. Los ejemplos que siguen son los del contrato oficial; **no implican que los endpoints ya existan**.

Parking Service

RUTAS PÚBLICAS

```
GET   /api/parking/spaces
GET   /api/parking/spaces/:id
POST  /api/parking/spaces
PATCH /api/parking/spaces/:id/status
```

CUERPOS DE SOLICITUD

```
POST /api/parking/spaces
{
  "code": "A-01",
  "level": "Nivel 1",
  "spaceType": "standard"
}

PATCH /api/parking/spaces/:id/status
{
  "status": "maintenance"
}
```

Las respuestas usan los campos `id`, `code`, `level`, `spaceType`, `status`, `createdAt` y `updatedAt` cuando corresponda. Ejemplo de consulta de plazas (corregido al formato oficial):

CONSULTA DE PLAZAS

```
GET /api/parking/spaces

200 OK
{
  "data": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "code": "A-01",
      "level": "Nivel 1",
      "spaceType": "standard",
      "status": "available"
    }
  ],
  "meta": { "total": 1 }
}
```

Reservation Service

RUTAS PÚBLICAS

```
POST  /api/reservations
GET   /api/reservations
GET   /api/reservations/:id
PATCH /api/reservations/:id/cancel
POST  /api/reservations/sessions/:reservationId/entry
POST  /api/reservations/sessions/:sessionId/exit
GET   /api/reservations/sessions/:sessionId
```

CREAR RESERVA

```
POST /api/reservations
Content-Type: application/json

{
  "spaceId": "550e8400-e29b-41d4-a716-446655440000",
  "vehiclePlate": "P123ABC"
}
```

En la ruta de entrada, el parámetro identifica la **reserva** (`reservationId`). El servicio crea la sesión y devuelve el `sessionId`. La entrada y la creación de la sesión deben ser una operación coherente, sin sesiones duplicadas. La salida devuelve los datos de la sesión: `sessionId`, `reservationId`, `spaceId`, `vehiclePlate`, `status`, `entryAt`, `exitAt` y `durationSeconds`.

Payment Service

COTIZACIÓN

```
POST /api/payments/quote
{
  "sessionId": "550e8400-e29b-41d4-a716-446655440000"
}

200 OK
{
  "data": {
    "quoteId": "550e8400-e29b-41d4-a716-446655440001",
    "sessionId": "550e8400-e29b-41d4-a716-446655440000",
    "durationSeconds": 3600,
    "billableHours": 1,
    "hourlyRateMinor": 1500,
    "amountMinor": 1500,
    "currency": "GTQ",
    "expiresAt": "2026-10-19T19:00:00.000Z"
  }
}
```

El ejemplo supone una tarifa de Q15.00 por hora; es ilustrativo y no establece la tarifa definitiva. **Política de redondeo del prototipo:** se cobra cada hora iniciada; cualquier fracción positiva de hora se redondea hacia arriba, con un mínimo de una hora para una sesión de duración positiva. La política debe ser única y documentada.

PAGO SIMULADO

```
POST /api/payments
Idempotency-Key: 550e8400-e29b-41d4-a716-446655440002
Content-Type: application/json

{
  "quoteId": "550e8400-e29b-41d4-a716-446655440001",
  "sessionId": "550e8400-e29b-41d4-a716-446655440000",
  "method": "simulated"
}
```

- La cabecera `Idempotency-Key` es **obligatoria**; reenviar la misma operación no debe generar un segundo pago.
- El servicio valida que la cotización corresponda a la sesión y que no esté vencida.
- La respuesta incluye `paymentId`, `sessionId`, `quoteId`, `amountMinor`, `currency`, `method`, `status` y `createdAt`.

API Gateway

- Rutas públicas: `/api/parking/*`, `/api/reservations/*`, `/api/payments/*` y `/health`.
- Conserva el formato de respuestas de los microservicios.
- Si falla la comunicación, devuelve el formato uniforme de error con el código HTTP adecuado (502, 503 o 504).

3.7. Rutas públicas frente a rutas internas

Servicio	Ruta pública (Gateway)	Ruta interna (microservicio)
Parking	`/api/parking/spaces`	`/spaces`
Parking	`/api/parking/spaces/:id/status`	`/spaces/:id/status`
Reservation	`/api/reservations`	`/reservations`
Reservation	`/api/reservations/:id/cancel`	`/reservations/:id/cancel`
Reservation	`/api/reservations/sessions/:id/...`	`/sessions/:id/...`
Payment	`/api/payments/quote`	`/payments/quote`
Payment	`/api/payments/session/:sessionId`	`/payments/session/:sessionId`

Las rutas internas son una **propuesta inicial** derivada de la planificación del equipo; el responsable del Gateway debe confirmar la reescritura de prefijos con cada responsable de servicio y documentarla. Lo oficial para el frontend son las rutas públicas.

4. Flujo oficial de integración

1. El frontend consulta las plazas mediante `GET /api/parking/spaces`.
2. El usuario selecciona una plaza y registra la reserva mediante `POST /api/reservations`.
3. Reservation Service solicita a Parking Service la validación de disponibilidad.
4. El sistema confirma la reserva y actualiza el estado de la plaza de manera segura.
5. Al ingresar el vehículo, Reservation Service crea una sesión con estado `active`.
6. Al salir, el servicio cierra la sesión con estado `completed` y calcula la duración.
7. El frontend solicita una cotización a Payment Service con `sessionId`.
8. Payment Service calcula el importe conforme a la tarifa y la política de redondeo.
9. El frontend registra el pago simulado usando `quoteId`, `sessionId` y `Idempotency-Key`.
10. El frontend muestra la confirmación final.

4.1. Flujo de salida y pago

Paso	Actor	Acción
1	Frontend	Solicita registrar la salida: `POST /api/reservations/sessions/:sessionId/exit`.
2	Reservation Service	Cierra la sesión y devuelve `sessionId`, `entryAt`, `exitAt` y `durationSeconds`.
3	Frontend	Solicita el cálculo del importe: `POST /api/payments/quote` con `sessionId`.
4	Usuario	Confirma el pago simulado.
5	Payment Service	Registra el pago (`POST /api/payments` con `Idempotency-Key`) y devuelve el resultado.
6	Frontend	Muestra la confirmación.

4.2. Fallos parciales

Las operaciones que actualicen una reserva, una sesión o una plaza deben contemplar fallos parciales. **No asumir que una petición HTTP fallida significa que la operación nunca se ejecutó.** Implementar comprobaciones de estado, reintentos seguros y compensaciones cuando sea necesario.

4.3. Flujo de prueba completo

> Consultar plazas → crear reserva → registrar entrada → registrar salida → calcular importe → registrar pago simulado.

5. Distribución de la base de datos

Esquema	Propietario	Datos
`parking`	Parking Service	Plazas y estado de las plazas
`reservations`	Reservation Service	Reservas y sesiones
`payments`	Payment Service	Tarifas, cotizaciones y pagos

Convenciones

- Tablas y columnas en `snake_case`; identificadores de recursos como UUID.
- Fechas almacenadas como `TIMESTAMPTZ`; importes como enteros en centavos.
- Estados almacenados con exactamente los valores definidos en este documento.
- Cada microservicio modifica únicamente sus propias tablas.
- Las referencias a recursos de otros servicios se hacen mediante UUID e interfaces HTTP, nunca modificando datos ajenos.

Ejemplos de nombres de tablas

`parking.parking_spaces` (ya existe), `reservations.reservations`, `reservations.parking_sessions`, `payments.tariffs`, `payments.payment_quotes` y `payments.payments`.

Tareas de base de datos

☐ Revisar la tabla existente `parking.parking_spaces` y adaptarla al contrato.

☐ Diseñar las tablas de reservas y sesiones.

☐ Diseñar las tablas de tarifas, cotizaciones y pagos.

☐ Definir las relaciones mediante identificadores UUID.

☐ Documentar el esquema de cada servicio.

☐ Mantener credenciales fuera del repositorio.

☐ Definir medidas contra reservas duplicadas (por ejemplo, restricciones únicas o bloqueo de filas).

☐ Definir medidas contra pagos duplicados (por ejemplo, restricción única sobre la clave de idempotencia).

> Los tres esquemas pertenecen al **mismo proyecto de Supabase**. La separación por esquemas es lógica: si todos los servicios usan la misma credencial PostgreSQL, **no hay aislamiento real de permisos**. Ese aislamiento requeriría credenciales y permisos diferenciados por servicio, algo que debe documentarse como limitación del prototipo. Los secretos no se suben a GitHub.

6. Infraestructura y tareas compartidas

- ☐ Revisar y completar `docker-compose.yml`.
- ☐ Configurar las variables de entorno (sin subir secretos al repositorio).
- ☐ Confirmar que los servicios se comuniquen correctamente dentro de los contenedores.
- ☐ Documentar instalación y ejecución en `README.md`.
- ☐ Probar la comunicación mediante el API Gateway.
- ☐ Preparar pruebas de errores y servicios no disponibles (502, 503, 504).
- ☐ Probar reservas concurrentes sobre una misma plaza.
- ☐ Probar reintentos de pago con la misma `Idempotency-Key`.
- ☐ Ejecutar el flujo completo del sistema.

Flujo de prueba

> Consultar plazas → crear reserva → registrar entrada → registrar salida → calcular importe → registrar pago simulado.

7. Orden recomendado de desarrollo

Fase	Actividad
1	Acordar los contratos API y los modelos de datos (sección 3).
2	Desarrollar en paralelo el frontend y los microservicios.
3	Probar individualmente cada microservicio y su conexión a PostgreSQL.
4	Integrar las rutas mediante el API Gateway.
5	Integrar el frontend.
6	Ejecutar pruebas completas y documentar resultados.

Este documento no fija fechas ni duraciones; el equipo las definirá según su calendario.

8. Entregables de cada responsable

Entregable	Frontend	Parking	Reservation	Payment	Gateway
Código funcional de su componente	✔	✔	✔	✔	✔
Endpoints documentados	—	✔	✔	✔	✔
Ejemplos de solicitudes y respuestas	✔	✔	✔	✔	✔
Validaciones y manejo de errores	✔	✔	✔	✔	✔
Evidencia de pruebas	✔	✔	✔	✔	✔
Instrucciones para ejecutar su componente	✔	✔	✔	✔	✔

> El responsable de integración (API Gateway) debe verificar que las partes se comuniquen correctamente antes de dar por cerrada la integración.

9. Adaptaciones pendientes respecto al código actual

Estas son las diferencias conocidas entre lo previsto inicialmente y el contrato oficial. Hasta que se verifique el código, no se debe afirmar que alguna de ellas ya está resuelta.

Tema	Antes / posible estado actual	Debe quedar como
Tipo de plaza en JSON	`space_type`	`spaceType`
Listado de plazas	`total` + `spaces`	`data` (arreglo) + `meta.total`
Identificador de sesión en rutas	`/sessions/:id/...`	`/api/reservations/sessions/:sessionId/...`; en la entrada el parámetro es `reservationId`
Importe	`amount`, `total`, `price`	`amountMinor` (centavos)
Fechas de entrada y salida	Variantes como `entryDate` o `startTime`	`entryAt` y `exitAt` (ISO 8601 UTC)
Pagos	Sin control de reintentos	`Idempotency-Key` obligatoria
Errores	Formatos libres	Objeto `error` con `code`, `message`, `details`, `requestId`